

Autumn 22 solution

Answer to the question no 1:

1(a)

Greedy strategy does not give the optimal solution for 0/1 knapsack. The given statement is correct. The greedy strategy for 0/1 knapsack problem involves selecting items based on their individual values-to weight ratios and adding them to the knapsack until it is full or no more items can be added. The following examples will establish our statement.

Example: Let us consider that the capacity of the knapsack is $W = 8$ and the items are as shown in the following table.

Item	A	B	C	D
Profit	2	4	7	10
Weight	1	3	5	7

Using the greedy approach of 0-1 knapsack, the weight that's stored in the knapsack would be $A+B = 4$ with the maximum profit $2 + 4 = 6$. But, that solution would not be the optimal solution. Therefore, dynamic programming must be adopted to solve 0-1 knapsack problems.

1(b)

Symbols Frequencies

A — 3

B — 3

C — 26

D — 5

E — 3

F — 8

G — 13

H — 2

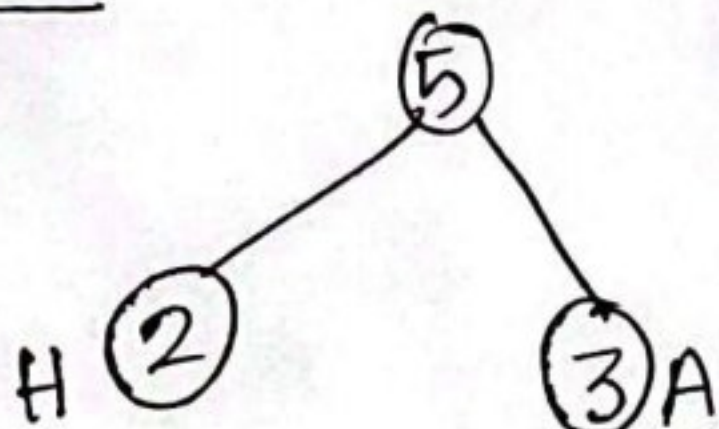
I — 16

J — 9

Step-1:

H A B E D F J G I C
(2) (3) (3) (3) (5) (8) (9) (13) (16) (26)

Step-2:



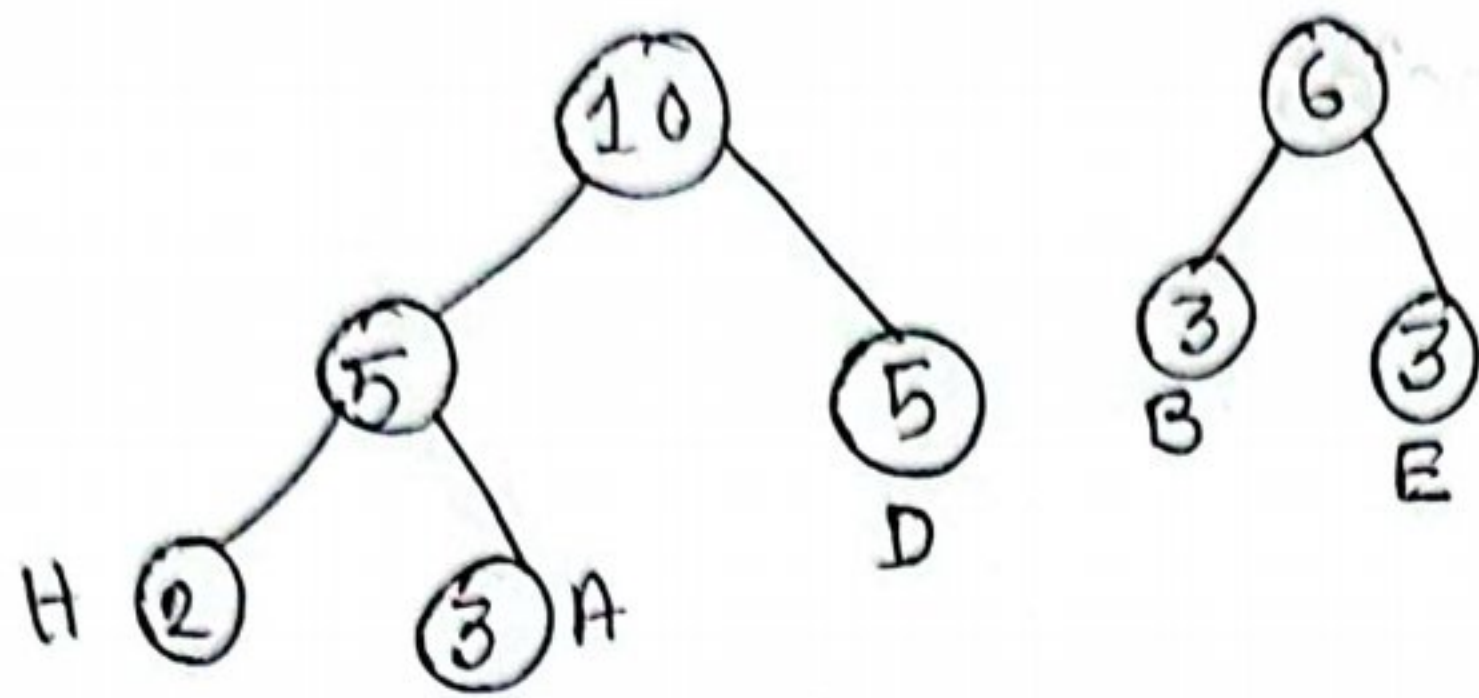
Step-3

Step-3:



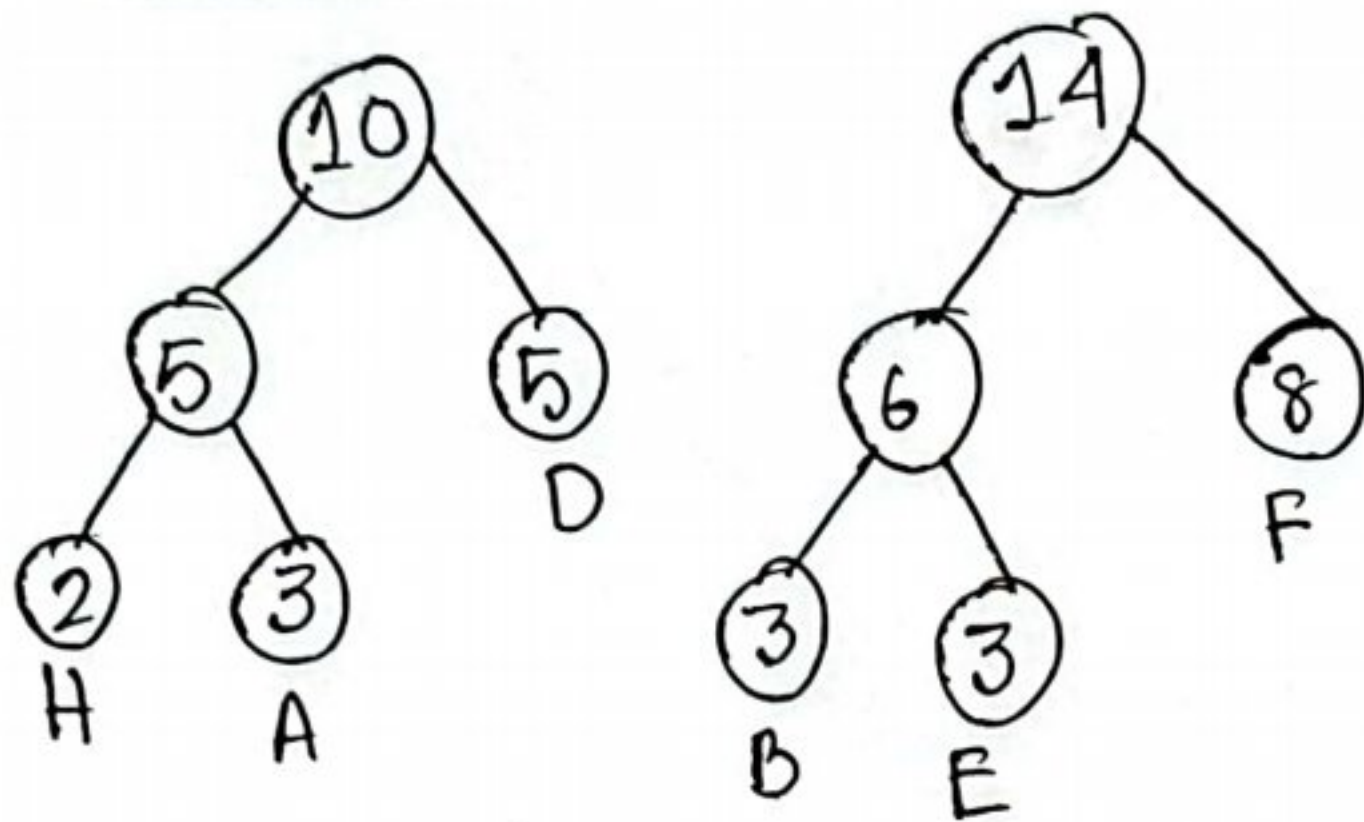
D F J G I C

Step-4:



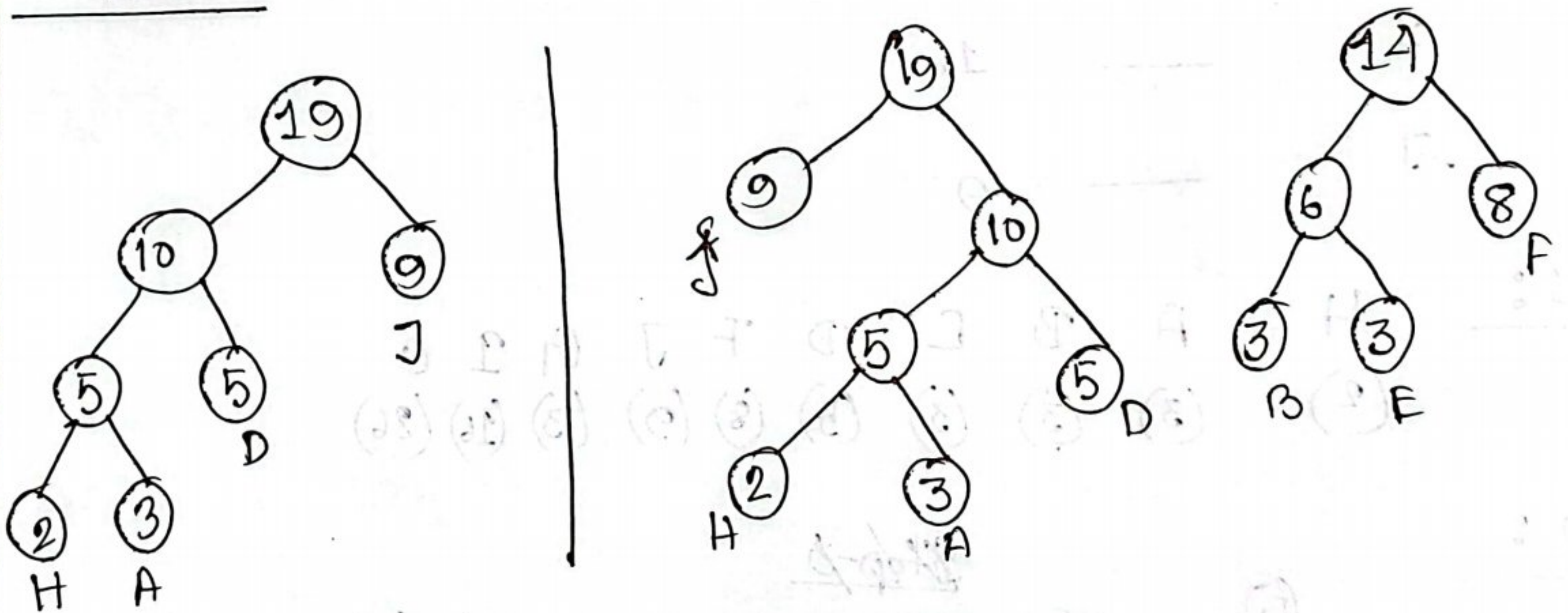
F 8 J 9 G 13 I 16 C 26

Step-5:

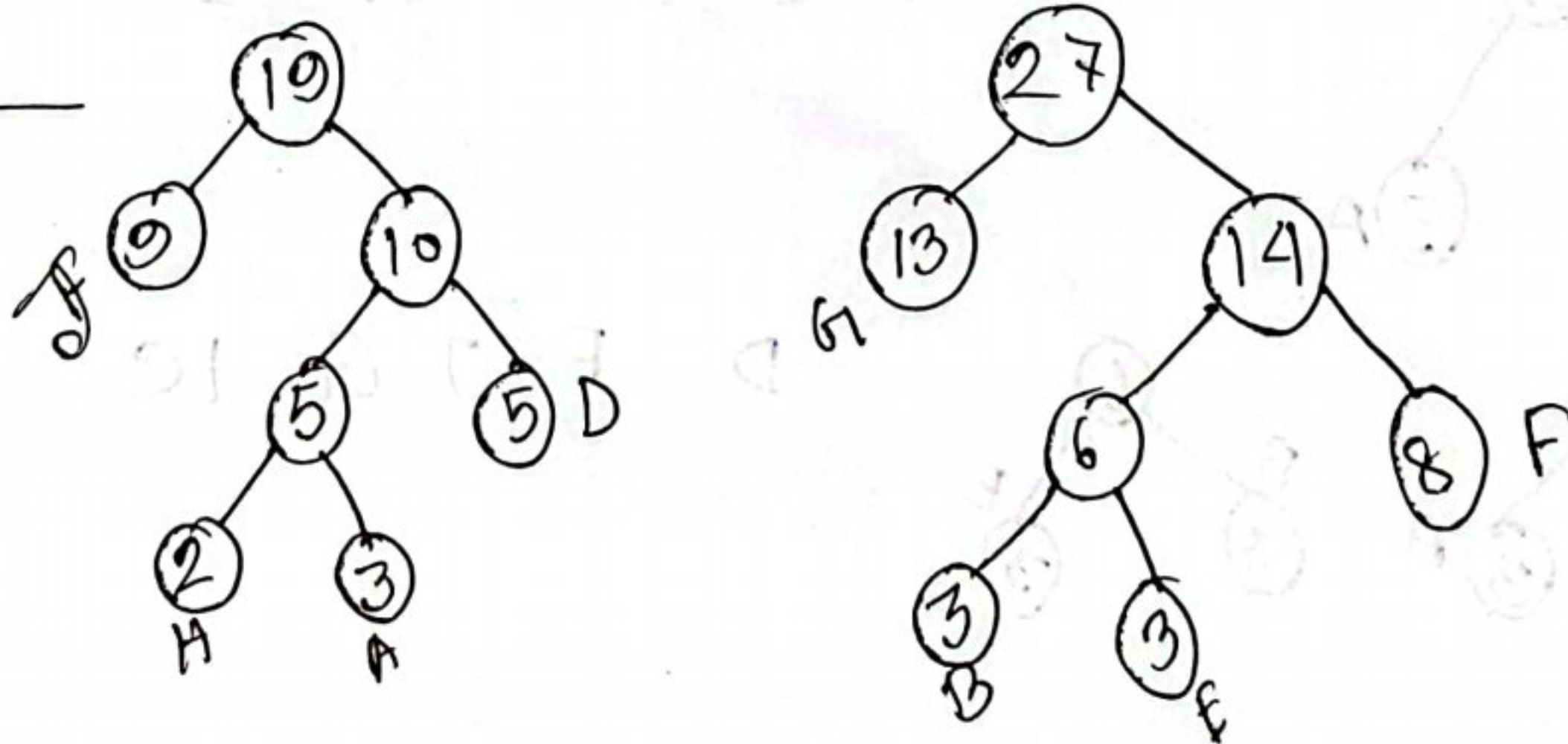


J 9 G 13 I 16 C 26

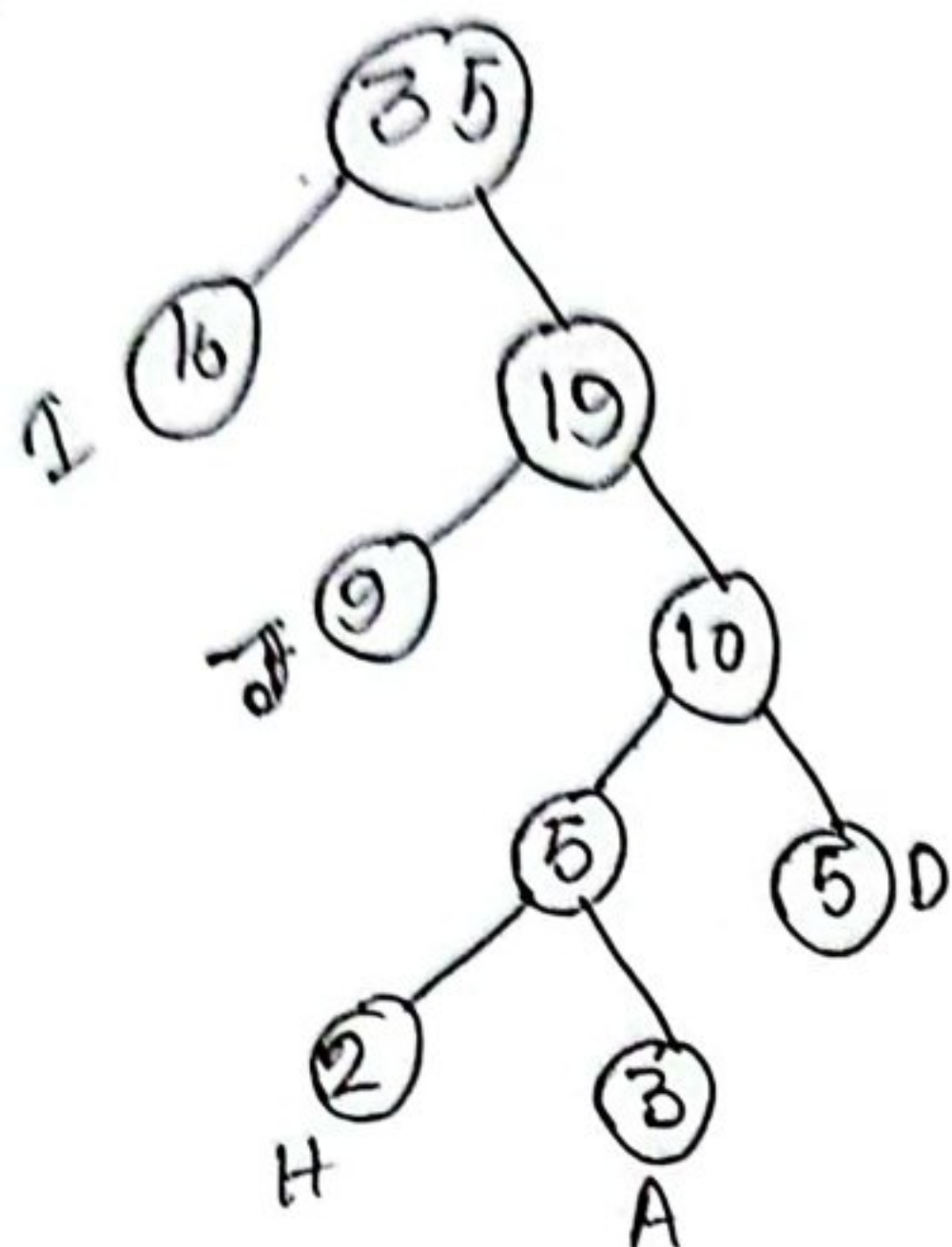
Step-6:



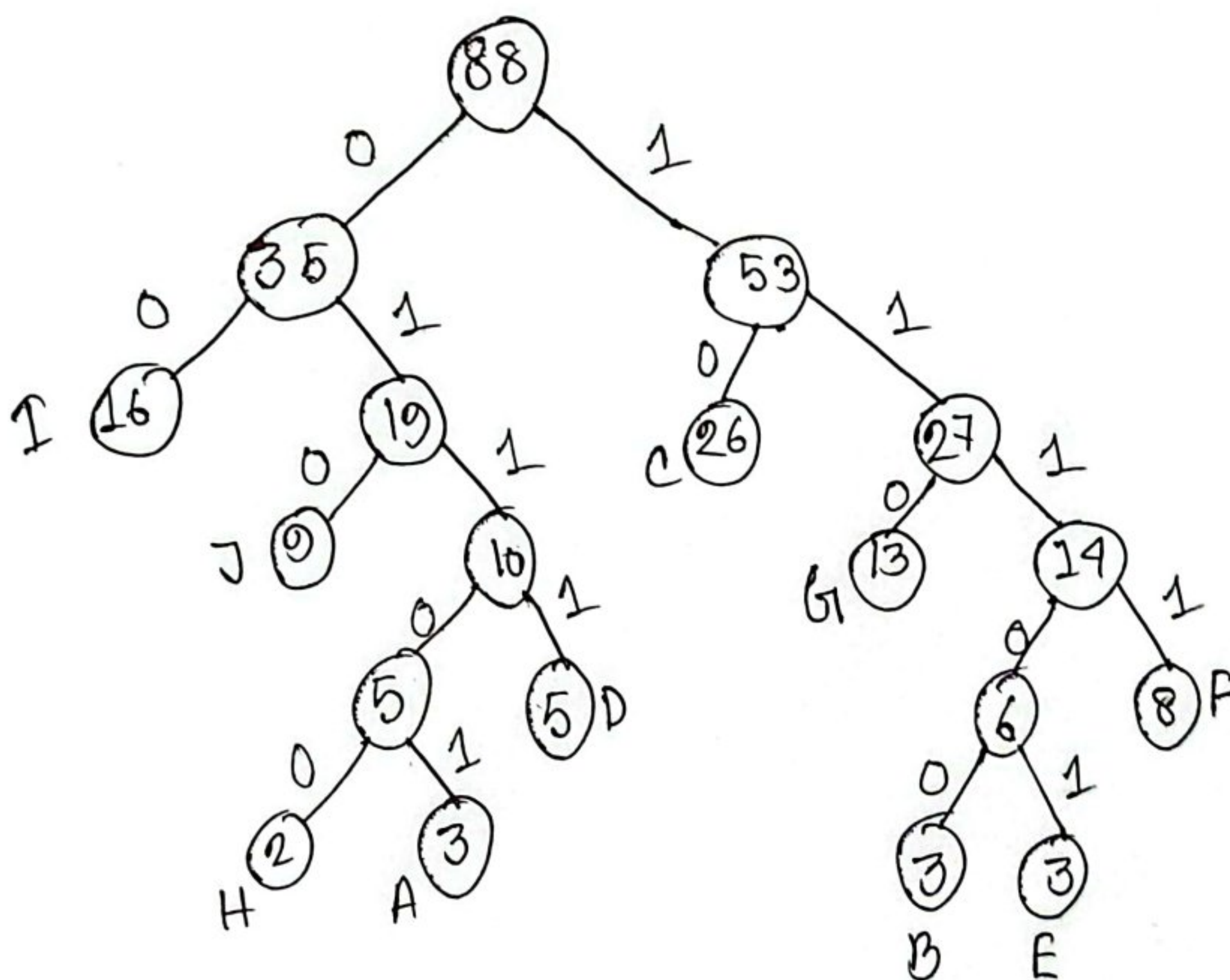
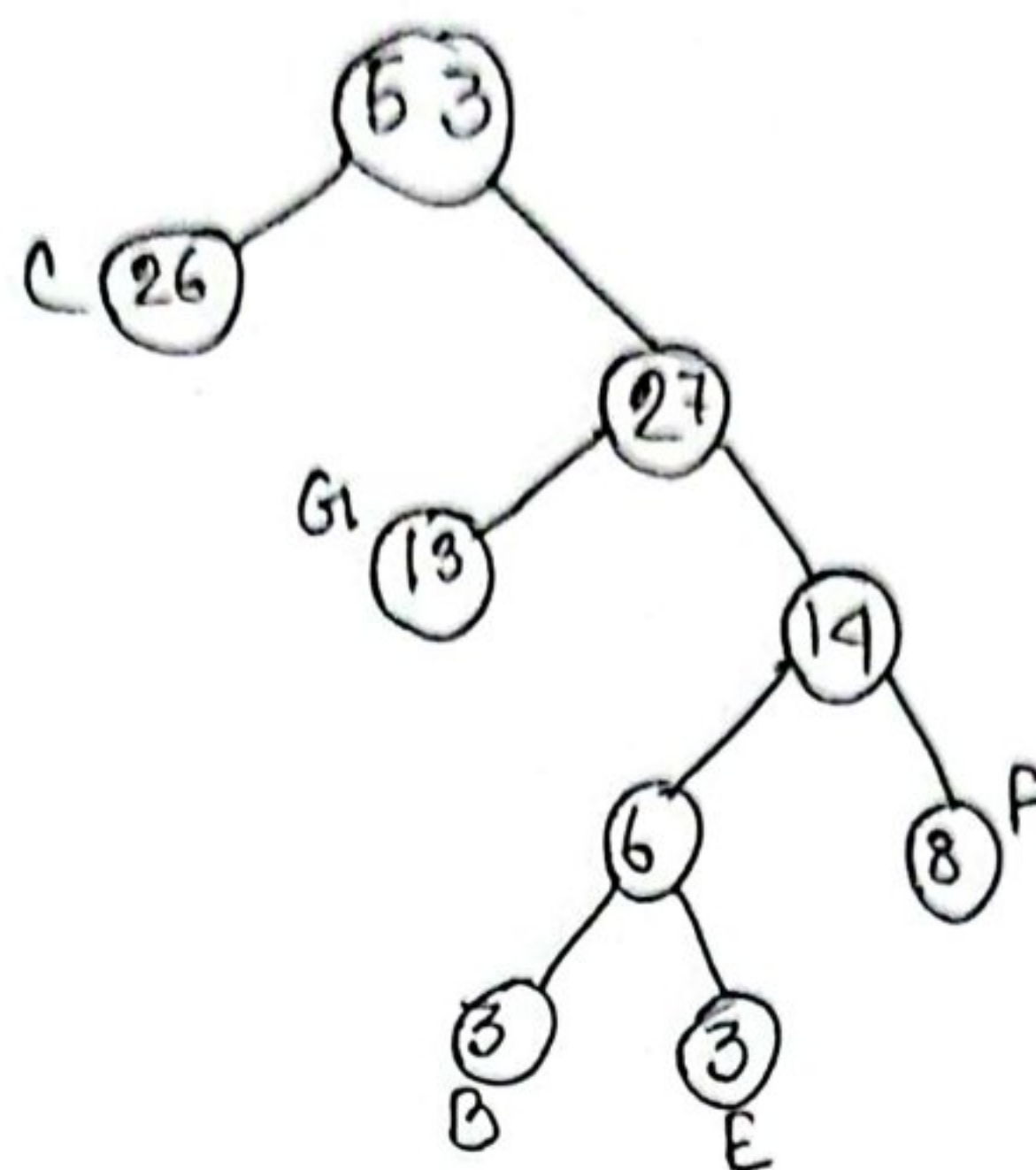
Step-7:



Step-8:



Step-9:



L-0
R-1

Encoding:

A → 01101

B → 11100

C → 10

D → 01111

E → 11101

F → 1111

G → 110

H → 01100

I → 00

J → 010

OR

1(b). Suppose we have a knapsack that has a weight limit w . There are items $i_1, i_2, \dots$ in each having $w_1, w_2, \dots, w_n$ and some benefit associated with $v_1, v_2, \dots, v_n$. We need to maximize benefit such that the total weight inside the knapsack is at most w . Demonstrate an algorithm to solve this problem.

The problem described is the classic 0/1 Knapsack problem, where we aim to maximize the total benefit while respecting the weight constraint of the knapsack. I'll outline a dynamic programming algorithm to solve this problem:

In this approach we'll define 2D DP of index for items defined on rows whereas weights from 1 to w on columns, and for every weight we can compute the answer for placing items till n^{th} item. Thus for every, $DP[i][j]$ we can calculate values for these two cases and store out the maximum of those two,

Therefore we can get answer for $\{i, j\}$ as,

$$DP[i][j] = \max[v[i] + DP[i-1][j - w[i]], DP[i-1][j]]$$

Pseudocode:

```
int knapsack 01(int w, int N, vector<int> dv, vector<int> tw)
{
    int DP[N+1][w+1]; // Defining DP
    for(int i=0; i<N+1; i++)
        DP[i][0] = 0; // If there are no items left, to N
        reaches 0 then DP[0][i] for every i will be 0.
```

```
for(int i=0; i<w+1; i++)
    DP[0][i]=0
```

```
for(int i=1; i<N+1; i++)
```

```
{
```

```
    for(int j=1; j<w+1; j++)
```

```
    {
```

```
        if(w[i-1] <= j)
```

```
        {
            DP[i][j] = max(v[i-1] + DP[i-1][j-w[i-1]], DP[i-1][j]);
        }
```

```
    }
```

```
    else{
```

```
        DP[i][j] = DP[i-1][j];
```

```
    }
```

```
}
```

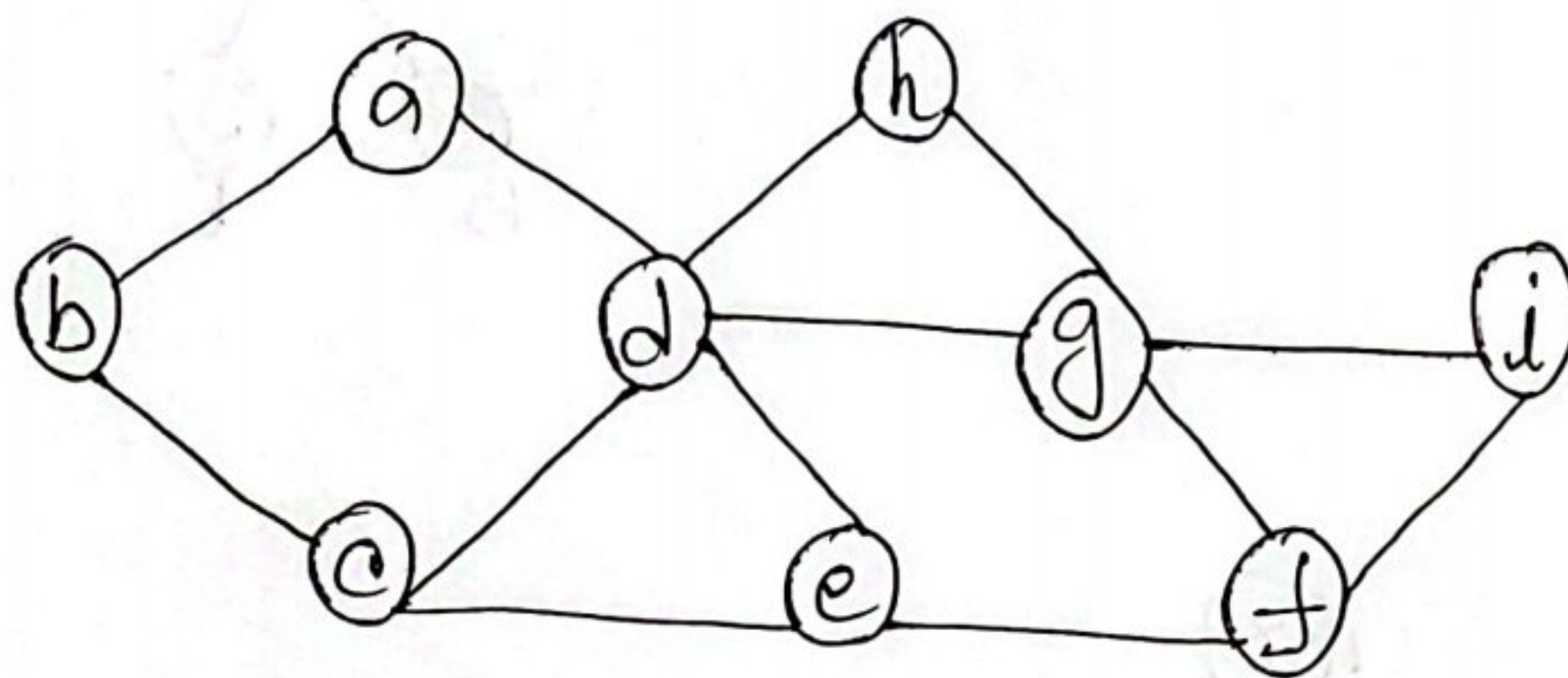
```
}
```

```
return DP[N][w];
```

Time complexity of 0/1 Knapsack is $O(nw)$ where n is the number of items and w is the capacity of knapsack.

2(a)

Drawing the undirected graph from given vertices,
nodes: a, b, c, d, e, f, g, h, i

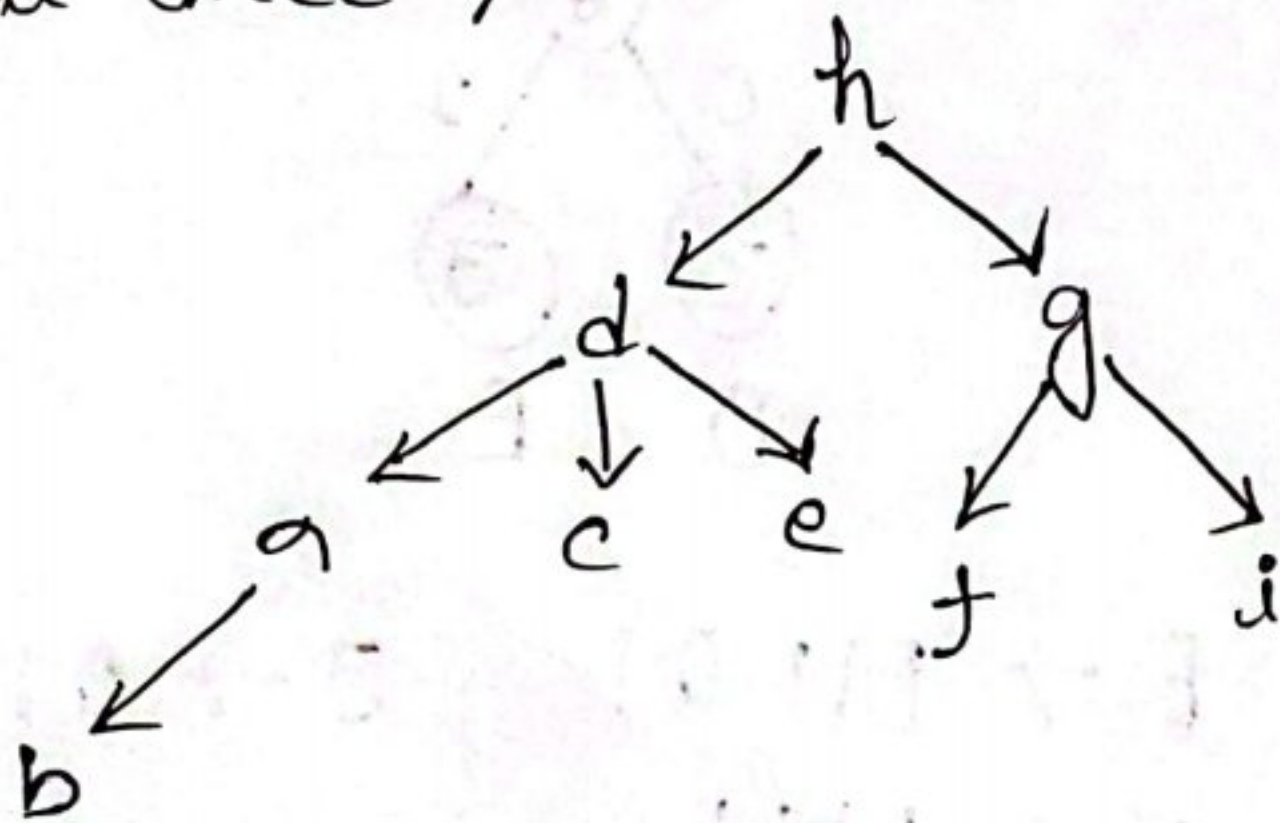


Traversing the graph using BFS starting from vertex h $\Rightarrow$

Queue: [h | d | g | a | c | ~~e~~ | f | i | b]

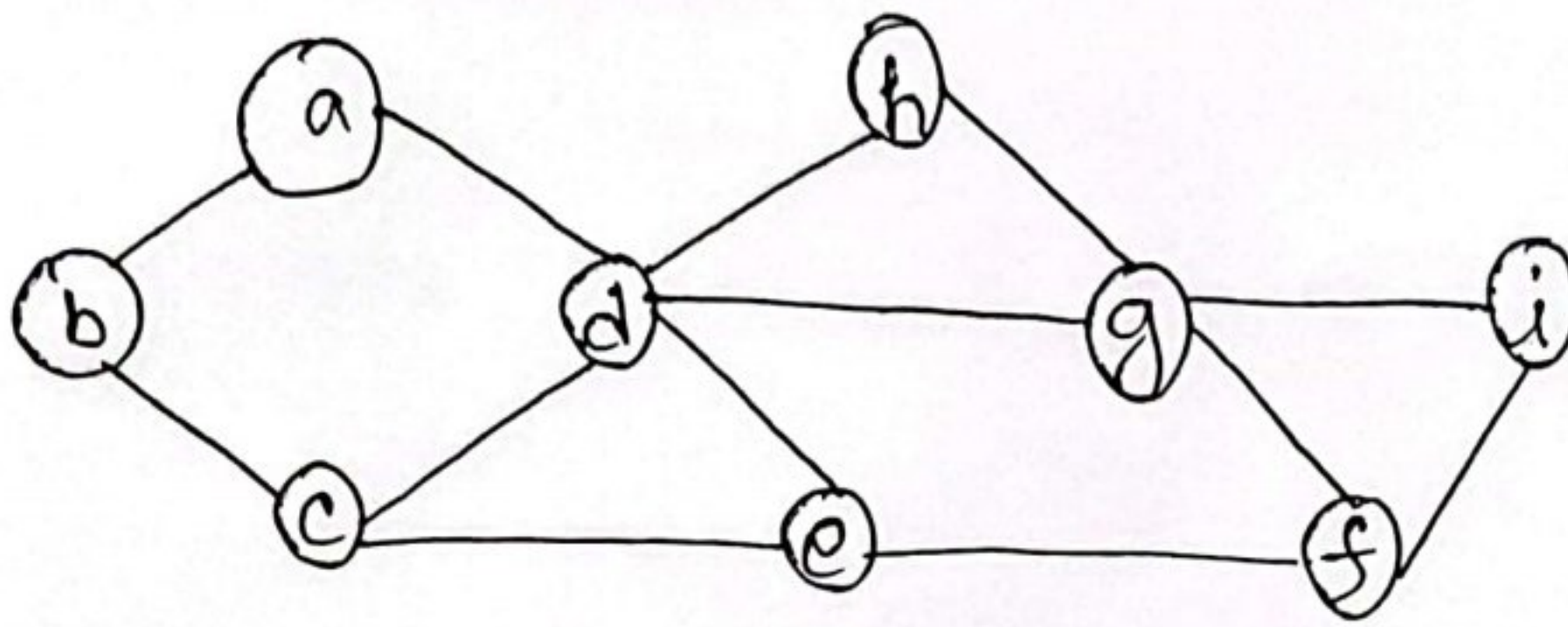
Output: h, d, g, a, c, ~~e~~, f, i, b

Final tree $\Rightarrow$

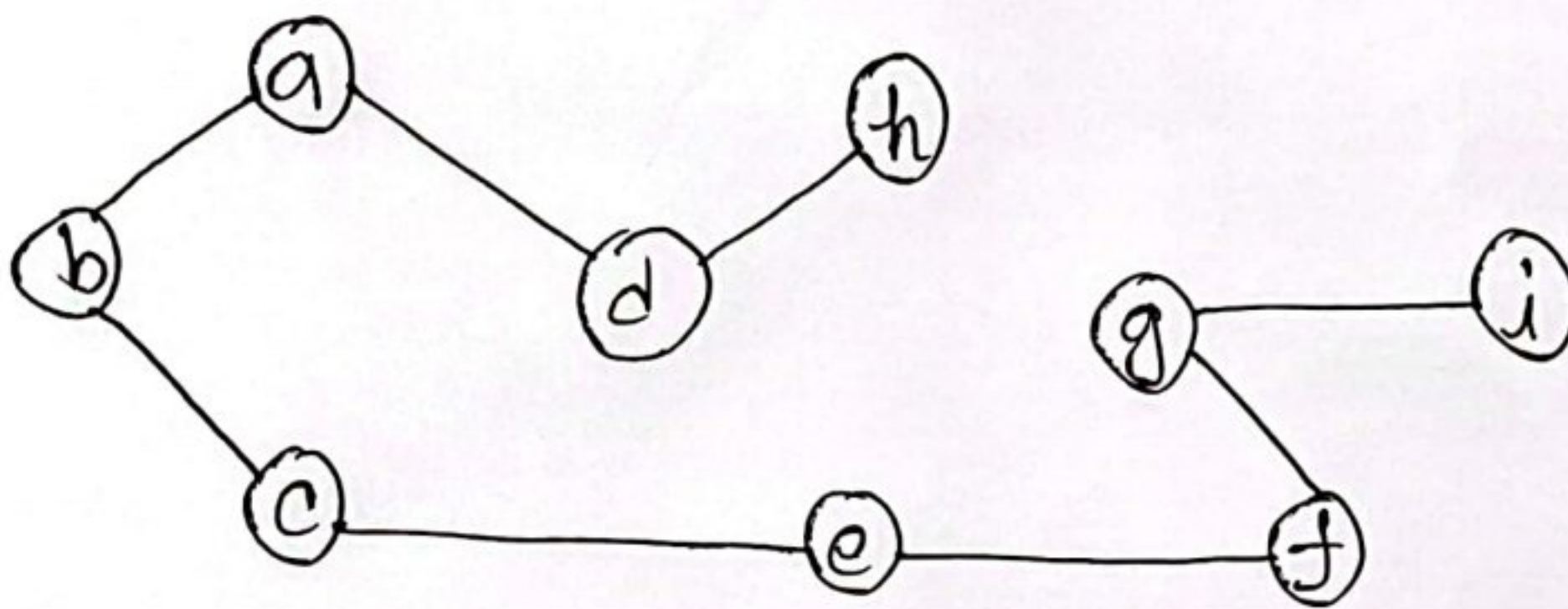


d values	π values
h = 0	h = none
d = 1	d = h
g = 1	g = h
a = 2	a = d
c = 2	c = d
e = 2	e = d
f = 2	f = g
i = 2	i = g
b = 3	b = a

2(a) or



Traversing the graph using DFS:



Output: ~~h, a, b, e~~, h, d, a, b, c, e, f, g, i

output	Discover Time, d	Finish Time f
h	0	17
d	1	16
a	2	15
b	3	14
c	4	13
e	5	12
f	6	11
g	7	10
i	8	9

Ans to the Q. NO-2(b)

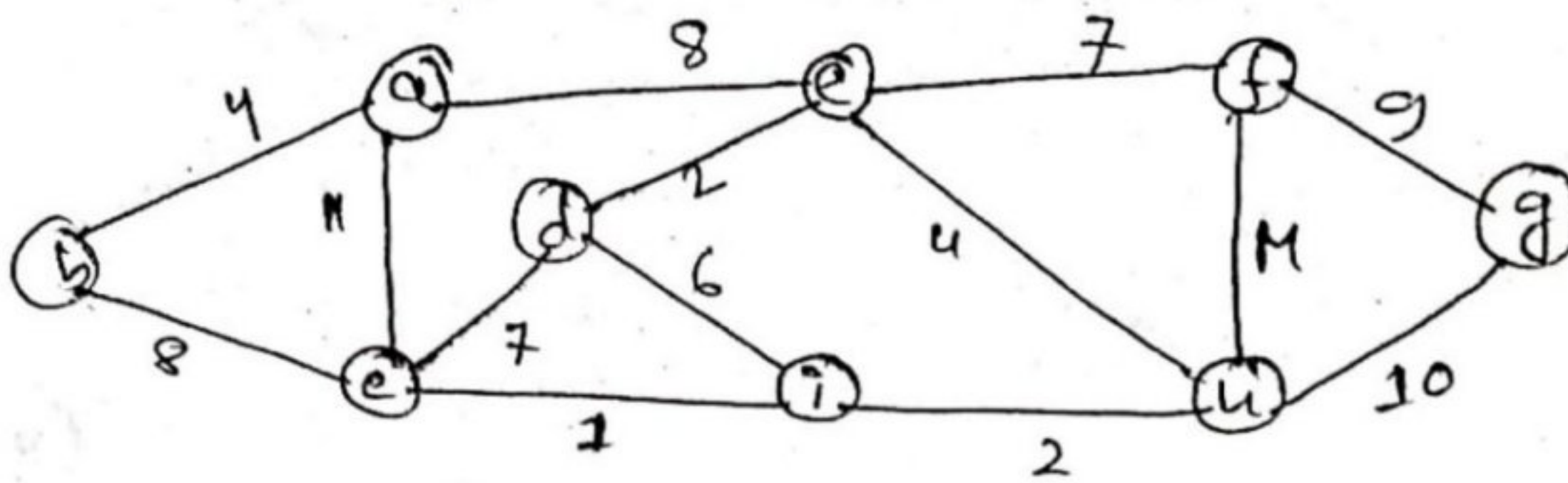
```
int IN, DEG [V+1];  
void compute InDeg (int V)  
{  
    int i, j;  
    for (i=1; i<=V; i++) {  
        for (j=1; j<=V; j++) {  
            if (graph[j][i] == 1) {  
                IN, DEG [i] ++;  
            }  
        }  
    }  
}
```

The in-degree of a vertex is equal to the number of times it appears in all the lists in Adj. if we search lists for each vertex, the time to compute the in-degree of every vertex $O(V|E|)$

Ans to the Q. NO - 2(c)

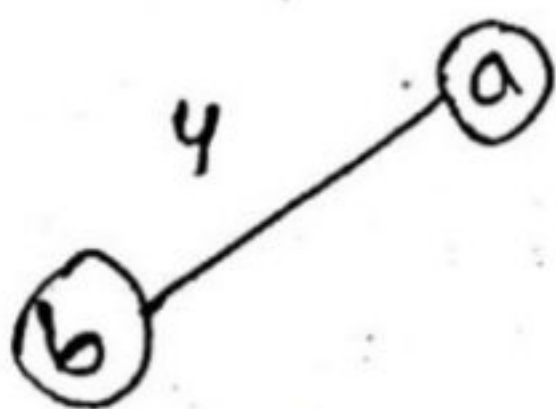
Minimum spanning Tree;

A minimum spanning tree is a subset of edges of a connected, edge-weighted undirected graph that connects all the vertices together, without any cycle with the minimum possible total edge weight.



	a	b	c	d	e	f	g	h	i
	0	0	0	0	0	0	0	0	0
T	0	4	8	e	4	7	9	21	216
P	0	a	b	c	h	e	f	i1	4d

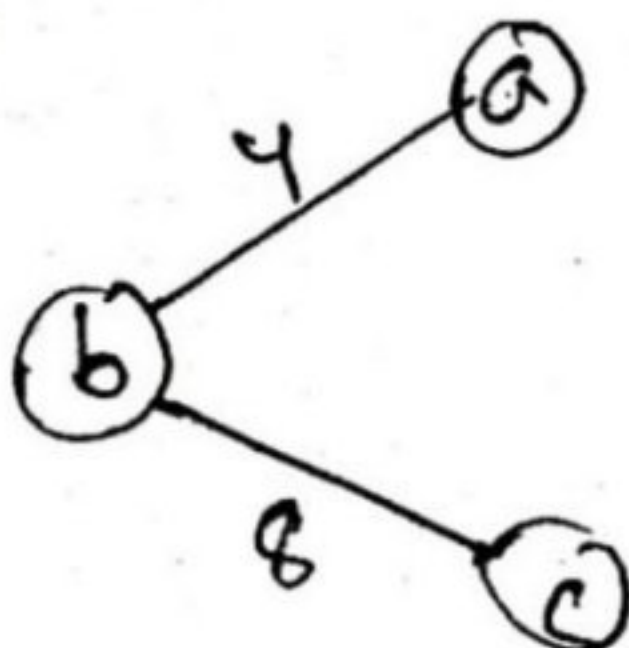
(1)



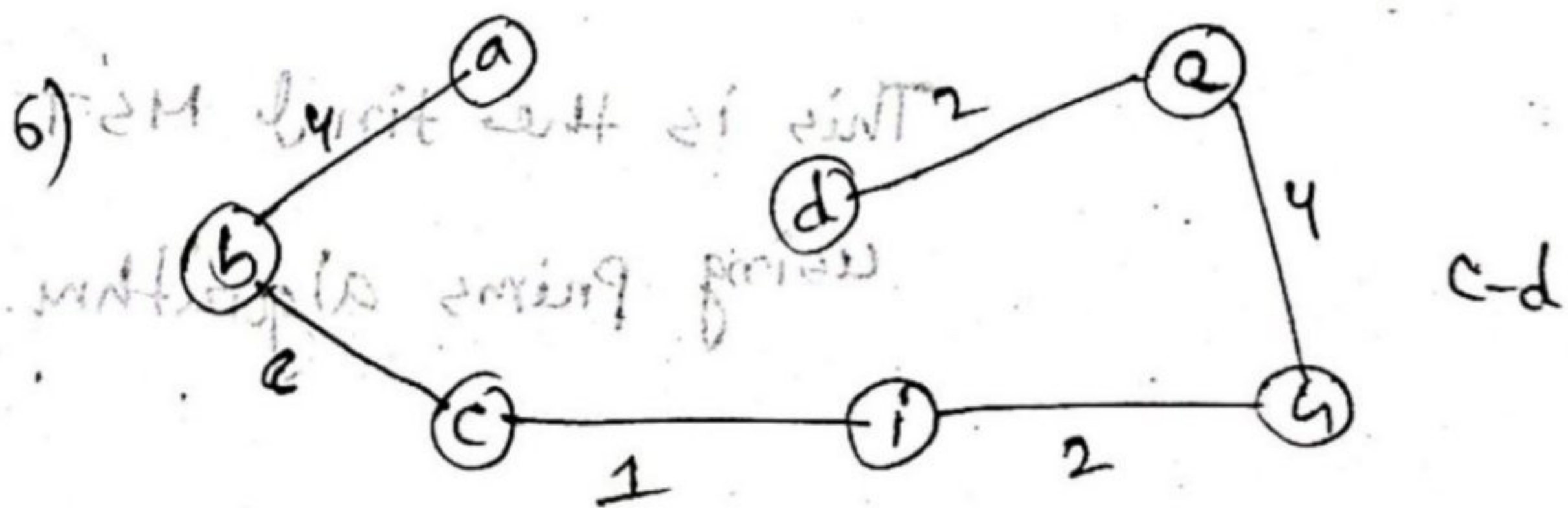
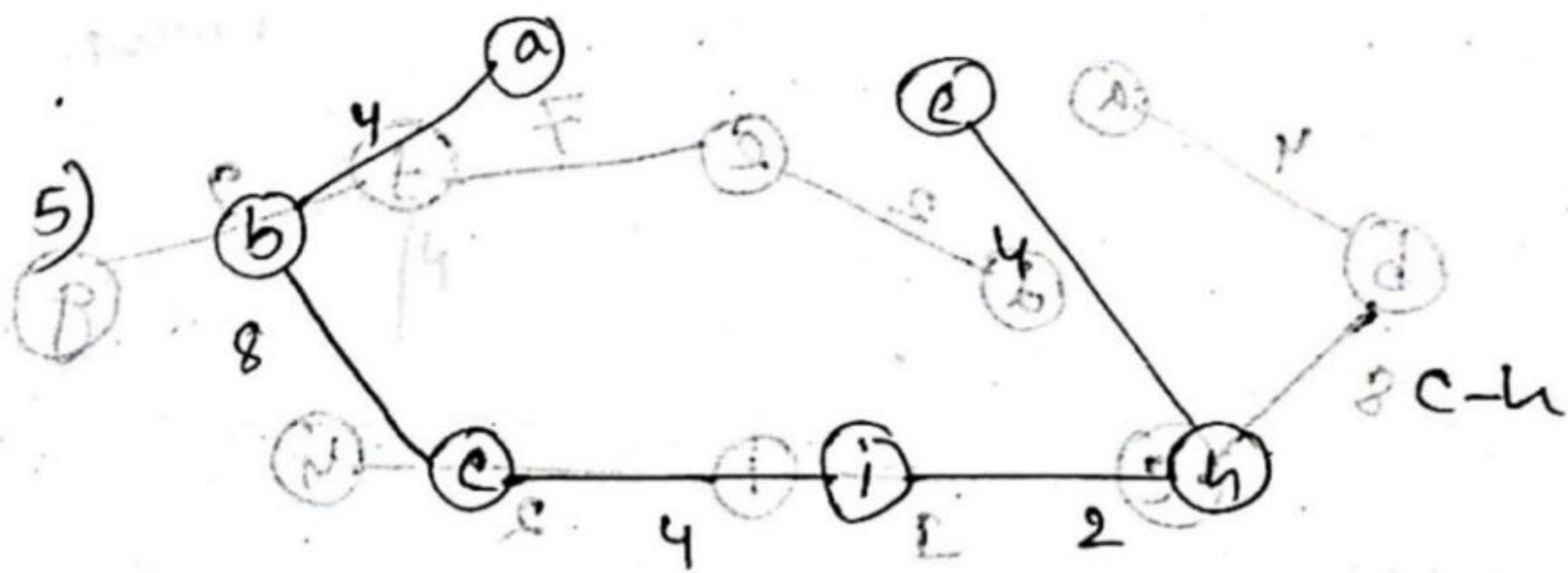
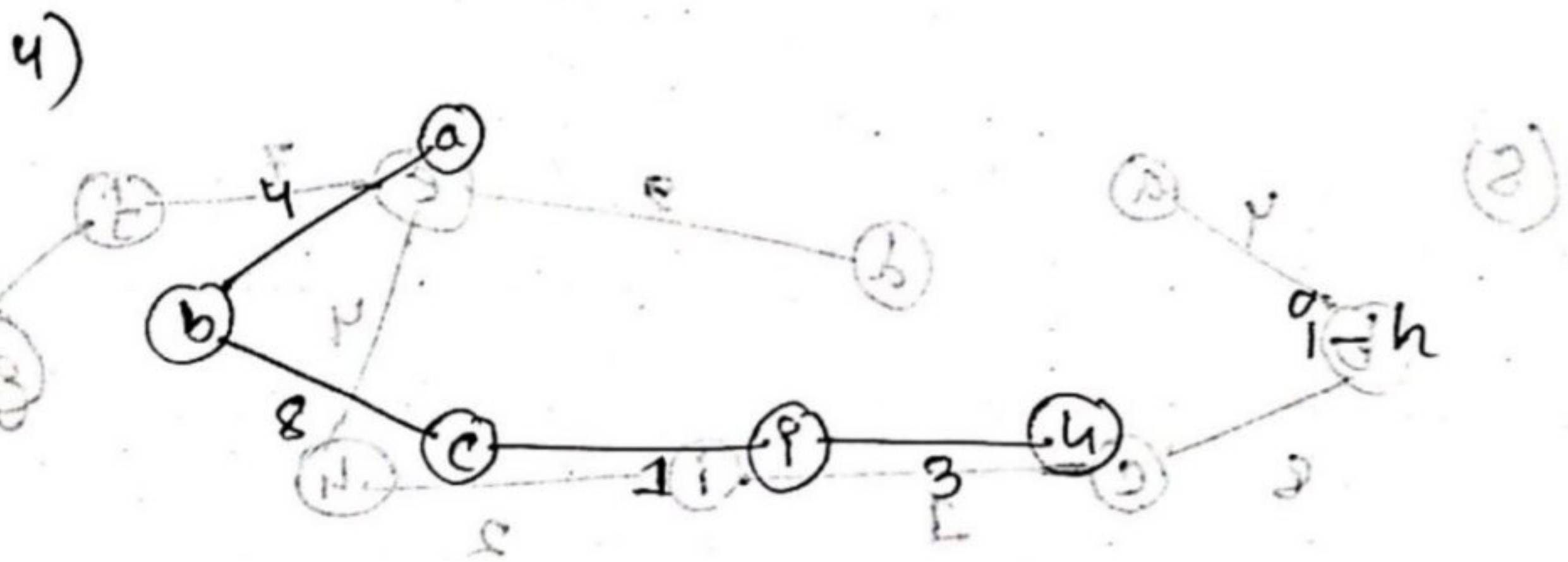
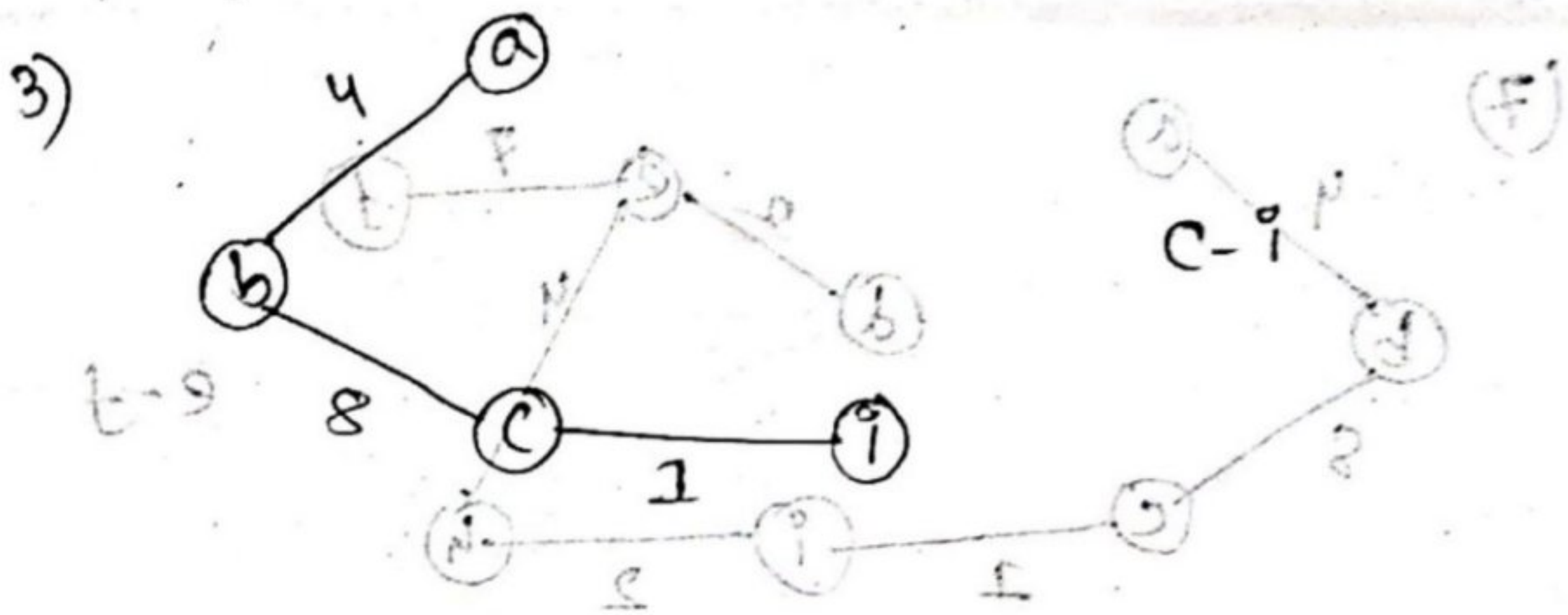
order of taking system

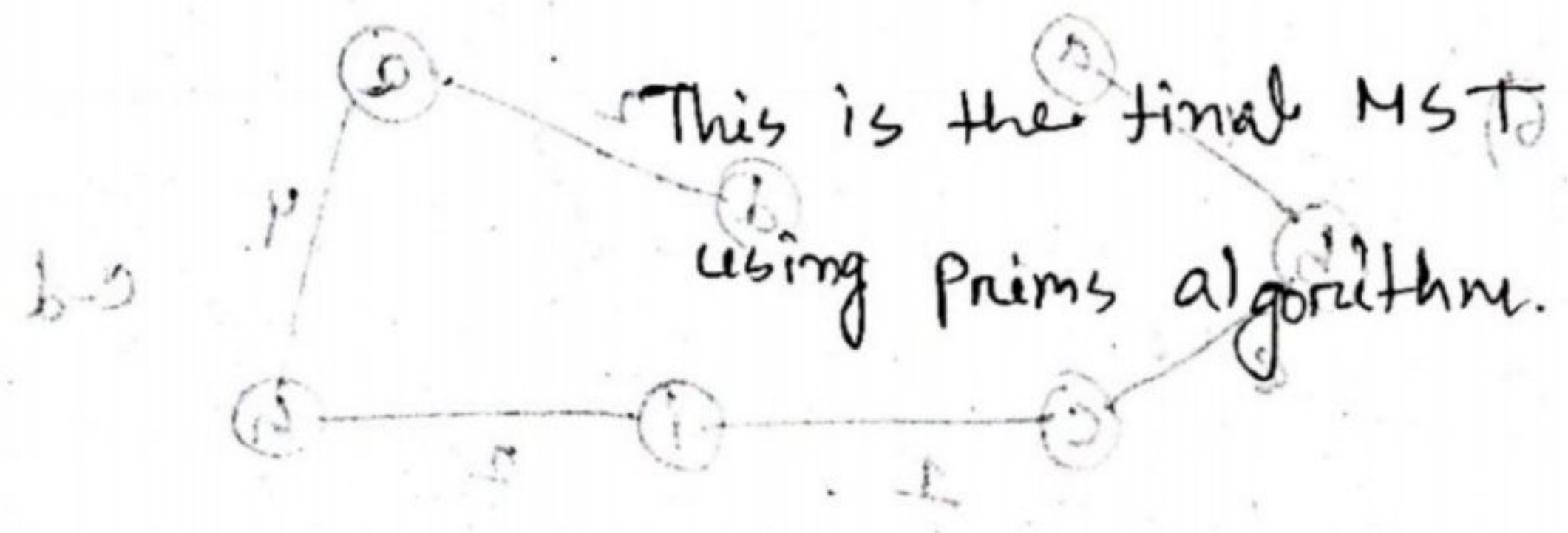
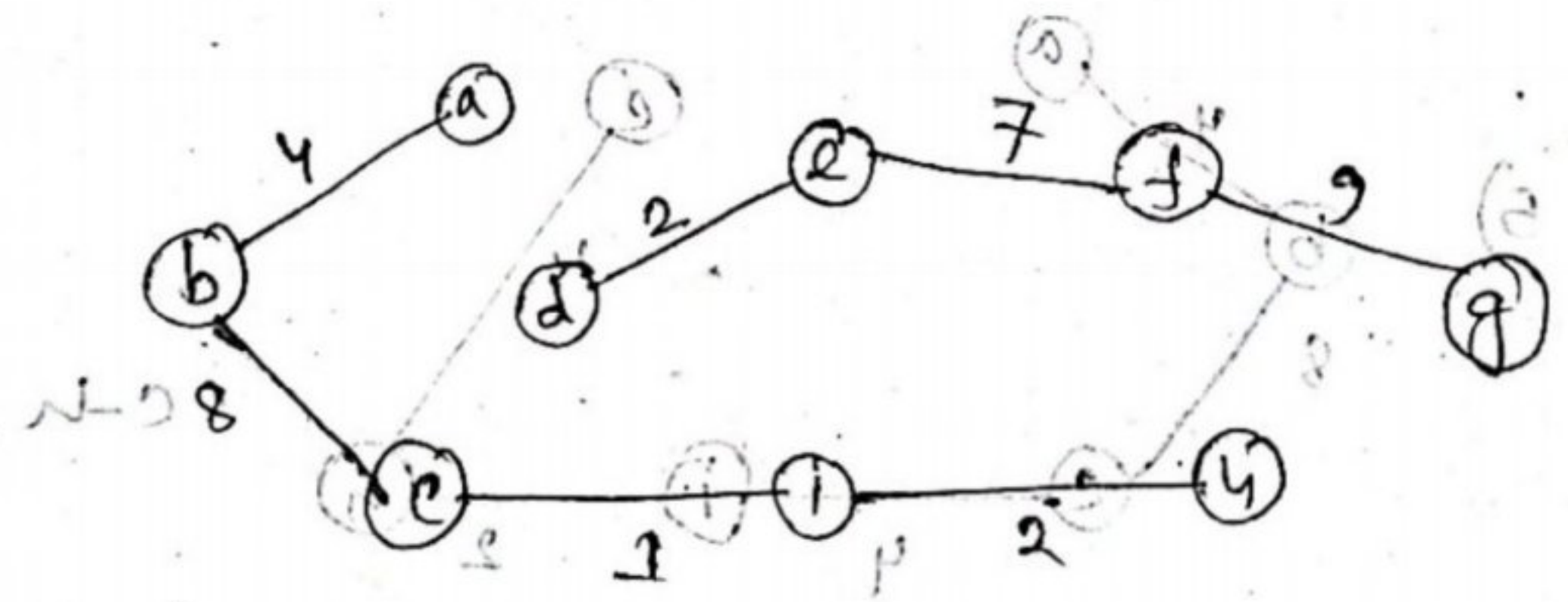
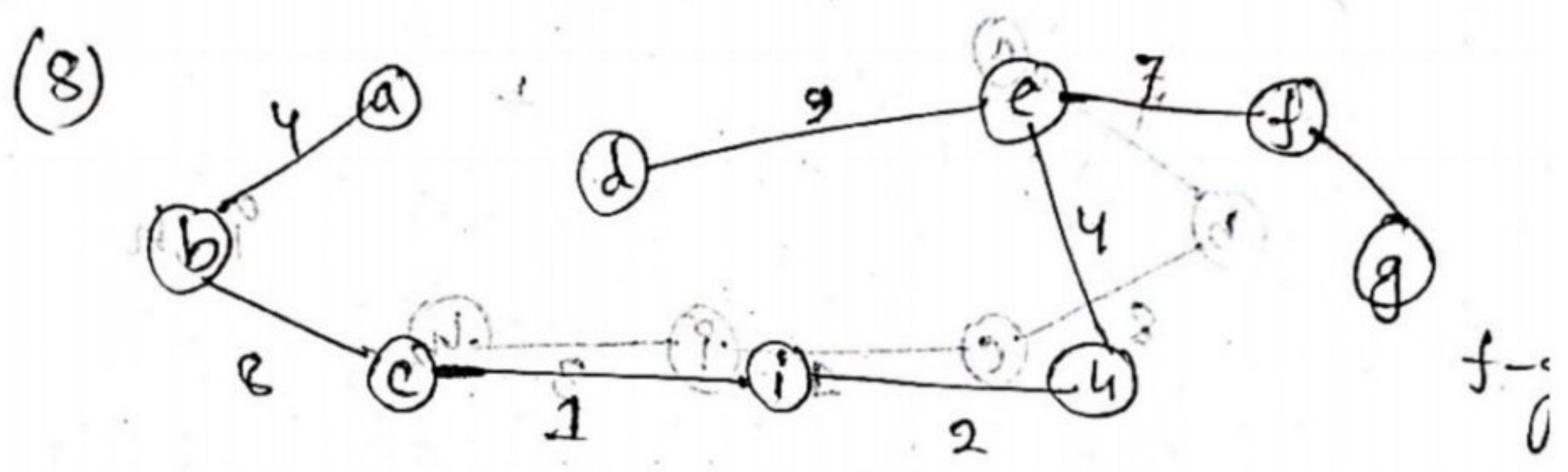
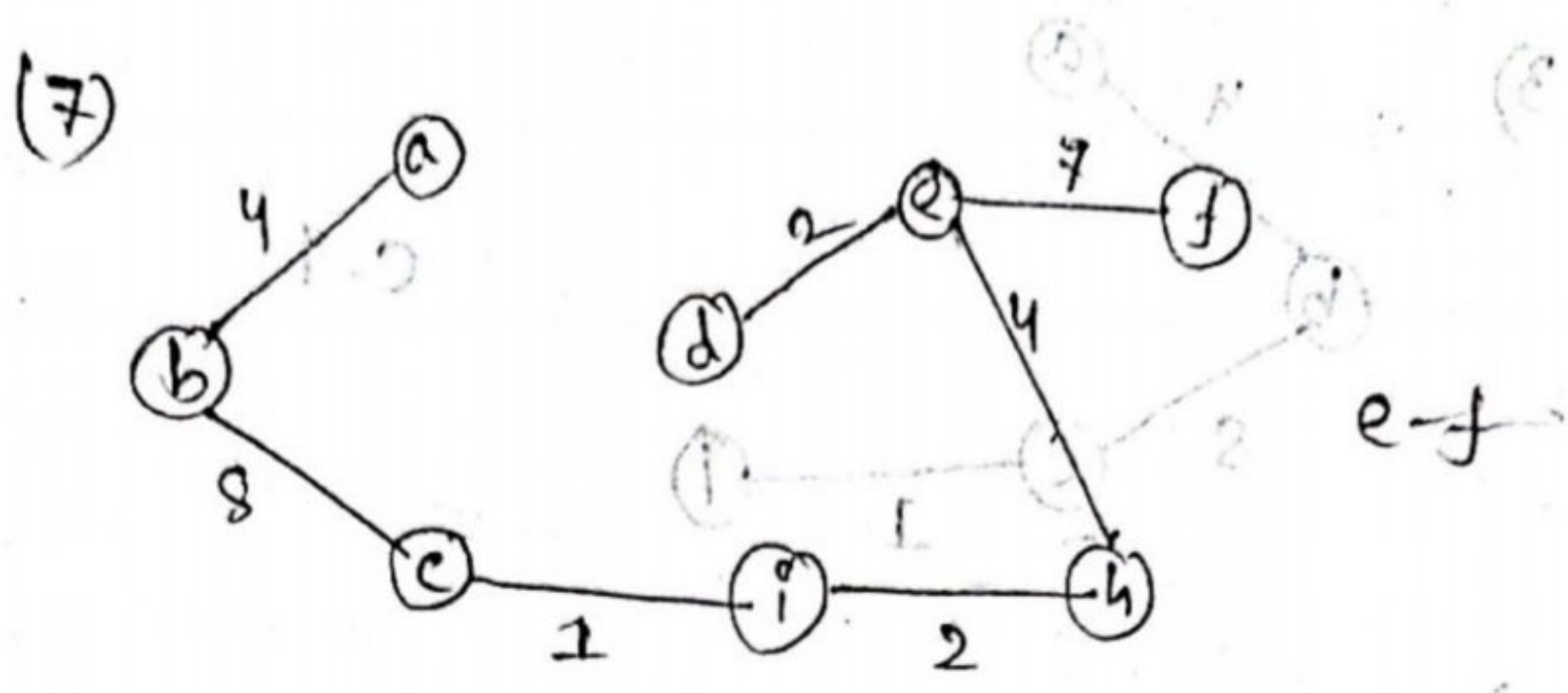
a-b

(2)

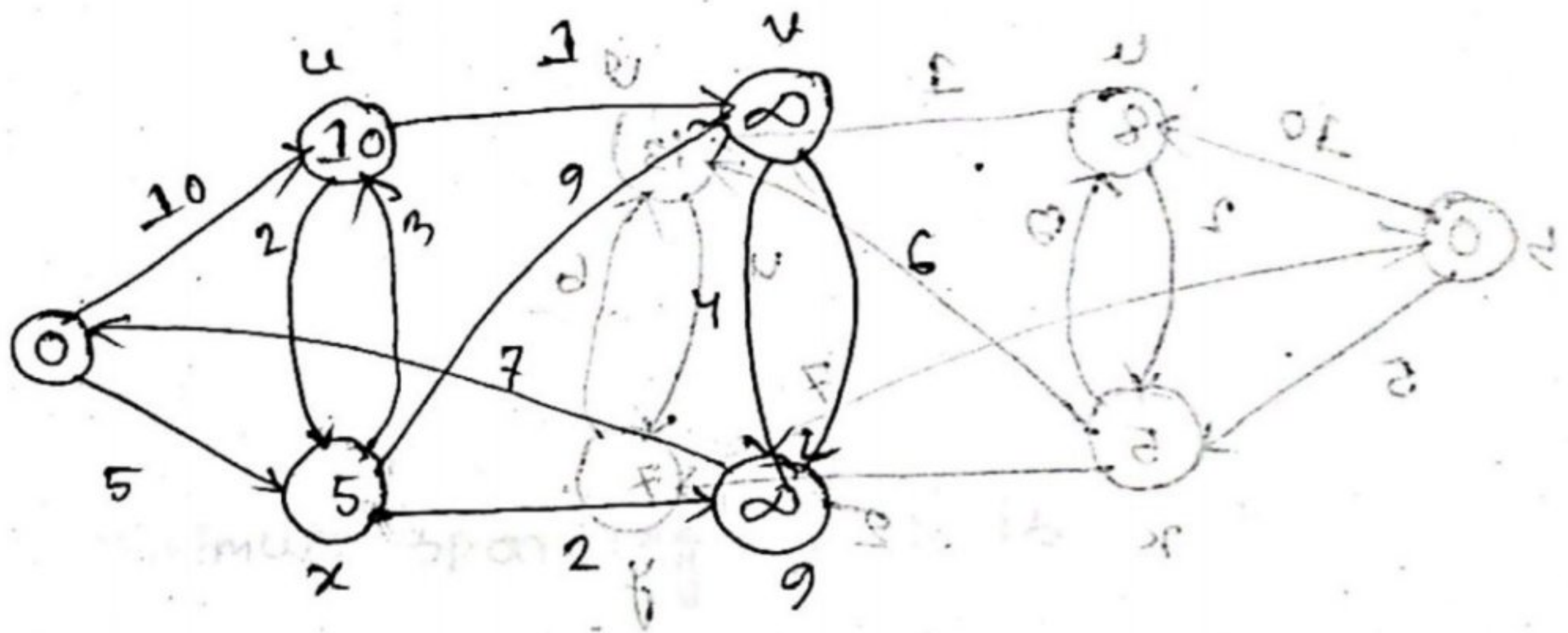
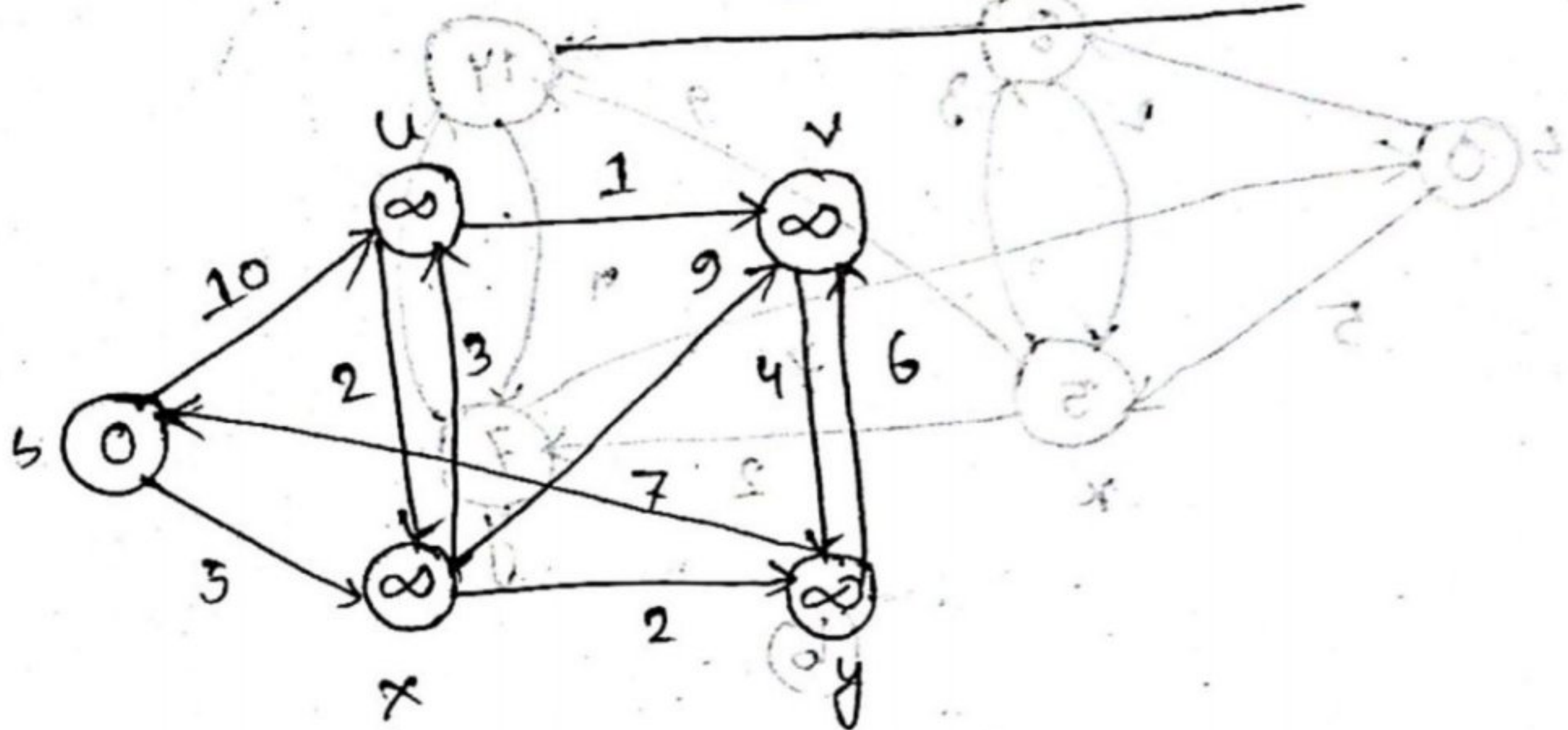


b-c



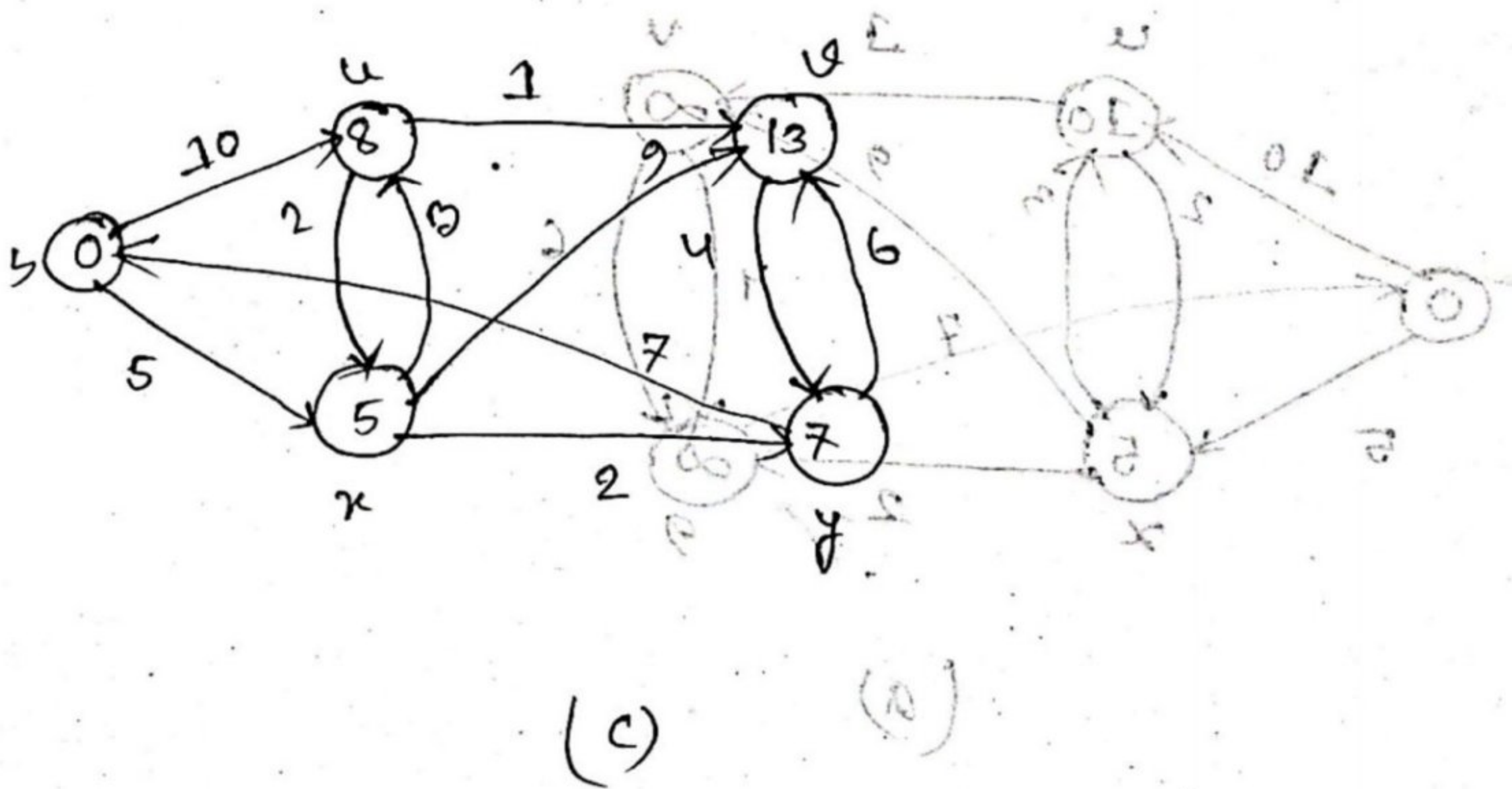
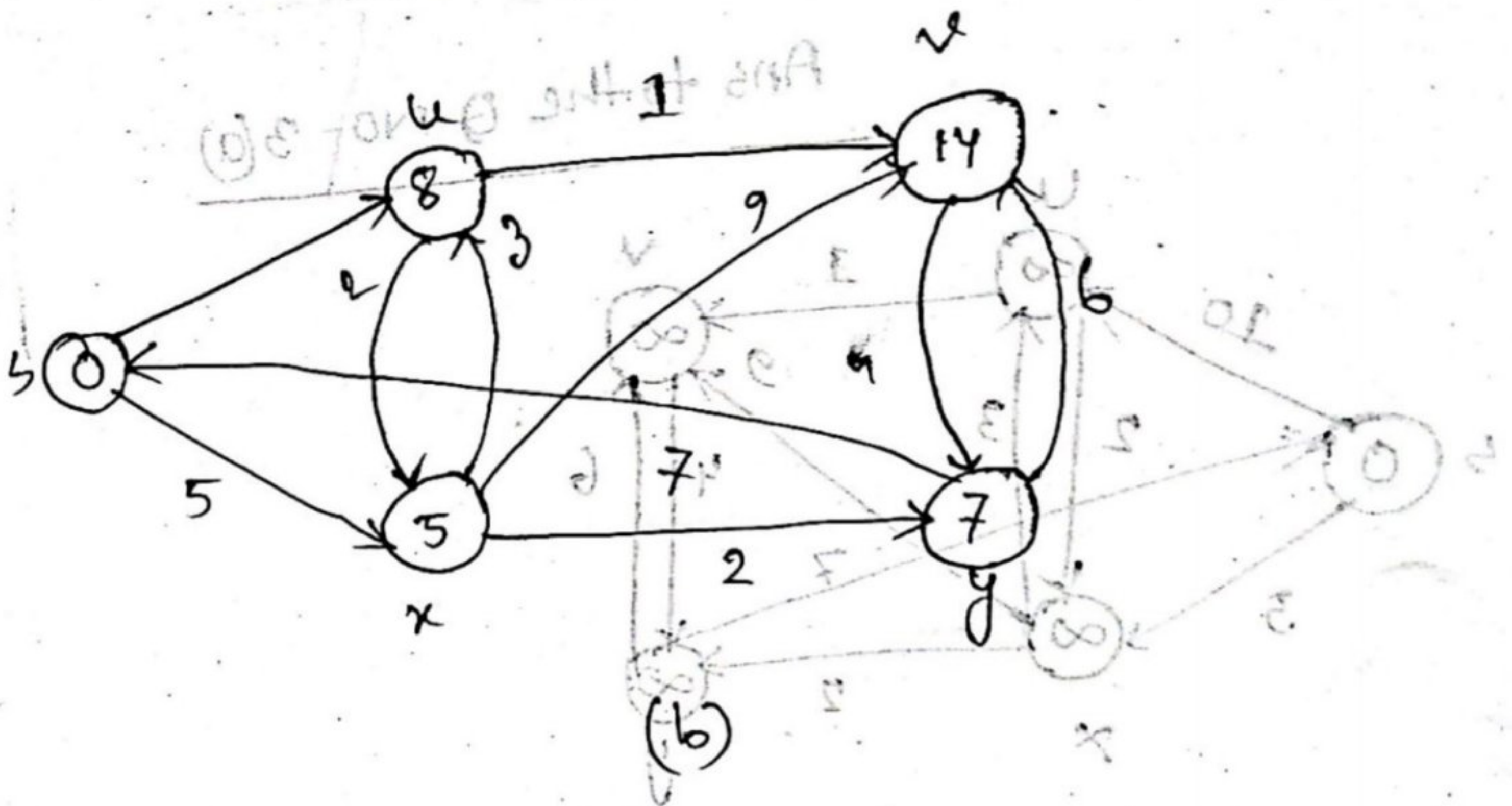


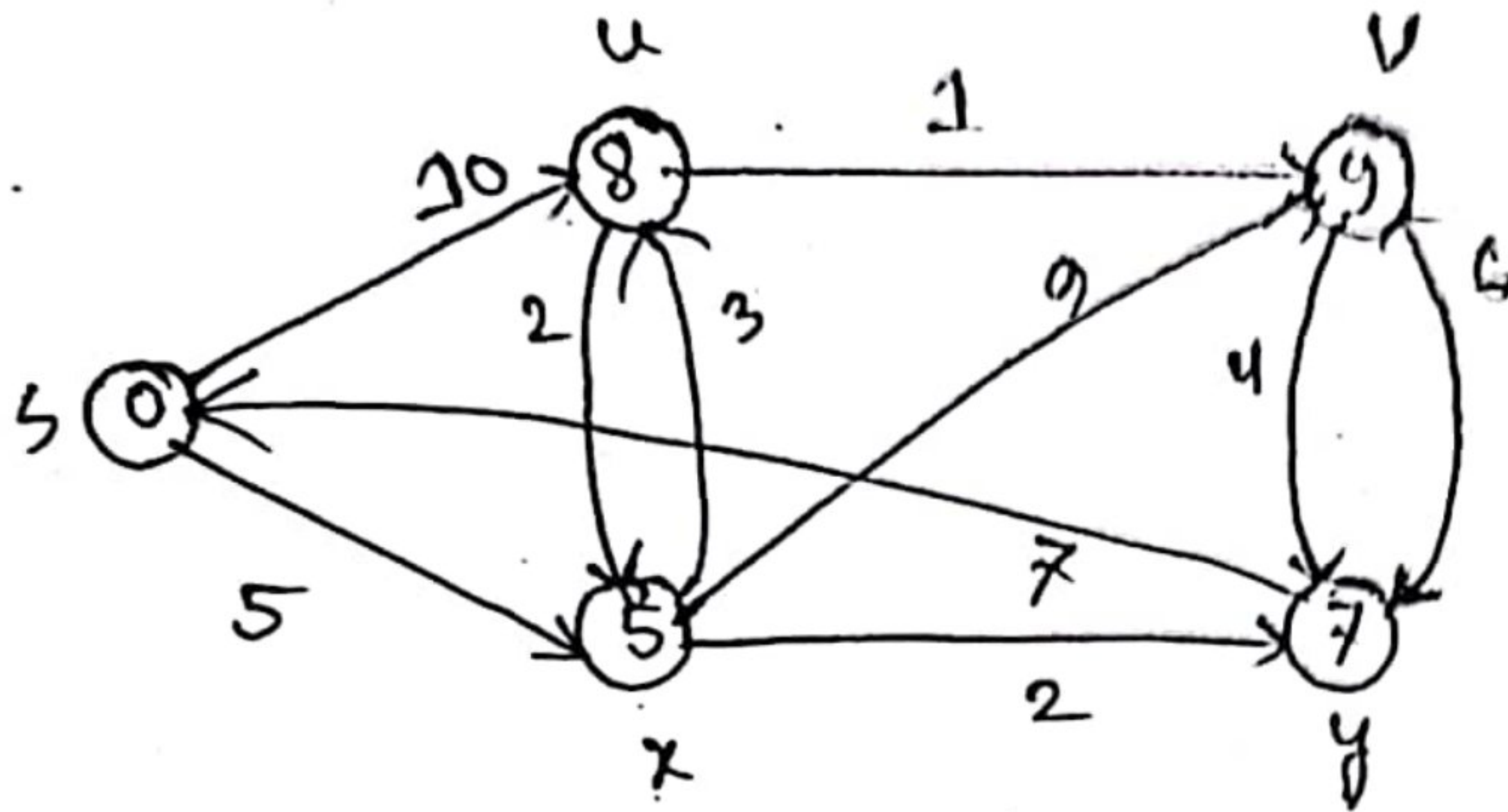
Ans to the Q. NO- 3(a)



(a)

(b)





(d)

The given matrix of $D(3)$

3(b)

$$D(3) = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & 7 & -4 \\ \text{inf} & 0 & \text{inf} & 1 & 7 \\ \text{inf} & 4 & 0 & 5 & 11 \\ 2 & -1 & 5 & 0 & -2 \\ \text{inf} & \text{inf} & \text{inf} & 6 & 0 \end{bmatrix} \end{matrix}$$

here,

$$D^3[1,2] < D^3[1,4] + D^3[4,2]$$

$$3 < 7 + (-1)$$

$$3 < 6$$

$$\therefore D^4[1,2] = 3.$$

$$D^3[1,3] < D^3[1,4] + D^3[4,3]$$

$$8 > 7 + (-5)$$

$$8 > 2$$

$$\therefore D^4[1,3] = 2.$$

$$D^3[1,5] < D^3[1,4] + D^3[4,5]$$

$$-4 < 7 + (-2)$$

$$-4 < 5$$

$$\therefore D^4[1,5] = -4.$$

$$D^3[2,1] > D^3[2,4] + D^3[4,1]$$

$$\inf > 1 + 2$$

$$\inf > 3$$

$$\therefore D^4[2,1] = 3.$$

$$D^3[2,3] > D^3[2,4] + D^3[4,3]$$

$$\inf > 1 + (-5)$$

$$\inf > -4.$$

$$\therefore D^4[2,3] = -4.$$

$$D^3[2,5] > D^3[2,4] + D^3[4,5]$$

$$7 > 1 + (-2)$$

$$7 > -1$$

$$\therefore D^4[2,5] = -1$$

$$D^3[3,1] > D^3[3,4] + D^3[4,1]$$

$$\inf > 5 + 2$$

$$\inf > 7$$

$$D^4[3,1] = 7$$

$$D^3[3,2] > D^3[3,4] + D^3[4,2]$$

$$4 > 5 + (-1)$$

$$\therefore D^4[3,2] = 4$$

$$D^3[3,5] > D^3[3,4] + D^3[4,5]$$

$$11 > 5 + (-2)$$

$$11 > 3.$$

$$\therefore D^4[3,5] = 3.$$

$$D^3[5,1] > D^3[5,4] + D^3[4,1]$$

$$\inf > 6 + 2$$

$$\inf > 8$$

$$\therefore D^4[5,1] = 8.$$

$$D^3[5,2] > D^3[5,4] + D^3[4,2]$$

$$\inf > 6 + (-1)$$

$$\inf > 5$$

$$\therefore D^4[5,2] = 5$$

$$D^3[5,3] > D^3[5,4] + D^3[4,3]$$

$$\inf > 6 + (-5)$$

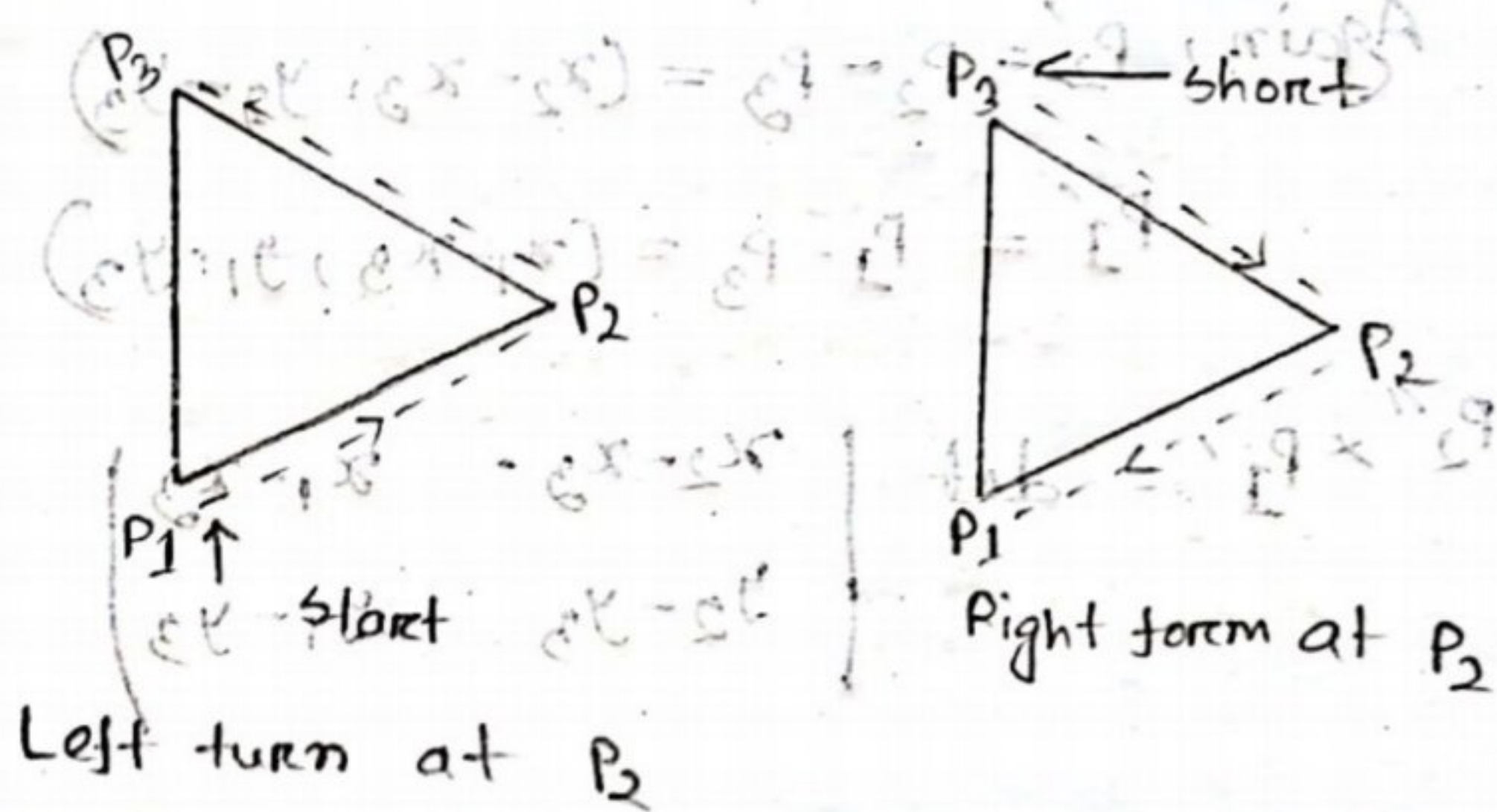
$$\inf > 1$$

$$\therefore D^4[5,3] = 1$$

$$\therefore D_4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 3 & 2 & 7 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix} \end{matrix}$$

Ans to the Q. No - 4(a)

Let $P_1(x_1, y_1)$, $P_2(x_2, y_2)$ and $P_3(x_3, y_3)$ are three points.



$$P_2' = P_2 - P_1 = (x_2 - x_1, y_2 - y_1)$$

$$P_3' = P_3 - P_1 = (x_3 - x_1, y_3 - y_1)$$

$$P_2' \times P_3' = \det \begin{vmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{vmatrix}$$

$$= x_2 y_3 - x_2 y_1 - x_1 y_3 + x_1 y_1 - x_3 y_2 + x_1 y_2 + x_3 y_1 - x_1 y_1$$

$$= (x_2 y_3 - x_2 y_1 - x_1 y_3 + x_1 y_2 + x_3 y_1 - x_3 y_2)$$

For $P_2 \times P_3$ we have got positive value which indicates that at point P_2 it turns left. When the order is P_1, P_2, P_3

$$\text{Again, } P_2' = P_2 - P_3 = (x_2 - x_3, y_2 - y_3)$$

$$P_1' = P_1 - P_3 = (x_1 - x_3, y_1 - y_3)$$

$$P_2'' \times P_1' = \det \begin{vmatrix} x_2 - x_3 & x_1 - x_3 \\ y_2 - y_3 & y_1 - y_3 \end{vmatrix}$$

$$= x_2 y_1 - x_3 y_1 + x_3 y_3 - x_2 y_3 - x_1 y_3 + x_1 y_2 + x_2 y_3 - x_3 y_2 + x_3 y_1$$

$$= (x_2 y_1 - x_3 y_1 + x_3 y_3 - x_2 y_3 - x_1 y_3 + x_1 y_2 + x_2 y_3 - x_3 y_2 + x_3 y_1)$$

for $P_2'' \times P_1'$ we have got negative value it indicates the right turn at point P_2 when the order is P_3, P_2, P_1

Ans to the Q. No - 4 (a) OR

(i) with respect to the origin (0,0):

For point $P_1(3,7)$

$$\begin{aligned}\text{Polar angle of } P_1 &= \tan^{-1} \left(\frac{7}{3} \right) \\ &= 66.801 \text{ degree}\end{aligned}$$

For point $P_2(9,2)$

$$\begin{aligned}\text{Polar angle of } P_2 &= \tan^{-1} \left(\frac{2}{9} \right) \\ &= 12.528 \text{ degree}\end{aligned}$$

(ii) with respect to the origin (0,0):

For point $P_1(3,7)$

$$\begin{aligned}\text{shifted coordinates: } P_1' &= (3-20, 7-0) \\ &= (-17, 7)\end{aligned}$$

$$\theta_1 = \left(\frac{7}{-17} \right)$$

$$= -22.380 \text{ degree}$$

For point $P_2(9, 2)$:

Shifted coordinates: $P_2' = (9 - 20, 2 - 0)$

$$= (-11, 2)$$

$$\theta_2 = \left(\frac{2}{-11} \right)$$

$$= -10.304 \text{ degree}$$

For point $P_3(2, 2)$:

$$\theta_3 = \left(\frac{2}{-18} \right)$$

(ii) with respect to origin $(0, 0)$:

For point $P_1(3, 4)$:

Shifted coordinates: $P_1' = (3 - 20, 4 - 0)$

$$= (-17, 4)$$

$$\theta_1 = \left(\frac{4}{-17} \right)$$

$$= -13.280 \text{ degree}$$

Ans to the Q. NO- 4(c)

Given a weighted, directed graph $G = (V, E)$ with weight function $w: E \rightarrow \mathbb{R}$, let $p = (v_0, v_1, \dots, v_k)$ be a shortest path from vertex v_0 to vertex v_k and for any i and j such that $0 \leq i \leq j \leq k$, let $p_{ij} = (v_i, v_{i+1}, \dots, v_j)$ be the subpath of p from vertex v_i to vertex v_j .

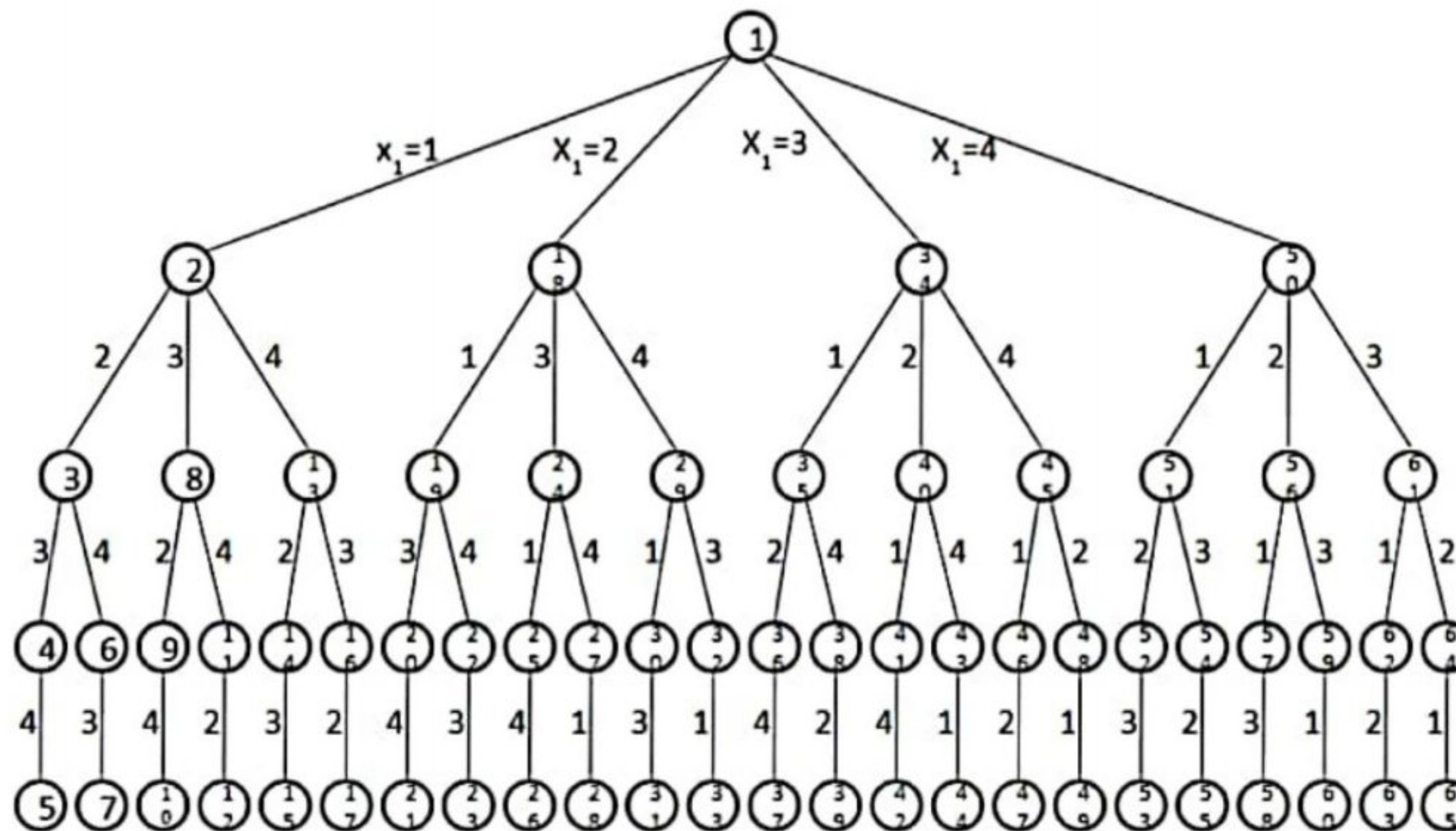
If we decompose path p into $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p_{ij}} v_j \xrightarrow{p_{jk}} v_k$, then we have that $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk})$. Now let us assume that there is a path p'_{ij} from v_i to v_j with weight $w(p'_{ij}) < w(p_{ij})$, then $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p'_{ij}} v_j \xrightarrow{p_{jk}} v_k$ is a path from v_0 to v_k whose weight $w(p_{0i}) + w(p'_{ij}) + w(p_{jk})$ is less than $w(p)$, which contradicts the assumption that p is a shortest path from v_0 to v_k .

5(a): Consider the 4-queen problem. Your task is to place the 4 queen in 4x4 chessboard so that no two queens attack each other.

- i. What is dead node in N-queen problem?
- ii. Design a state space tree which represents all possible arrangements for 4 non-attacking queens.

Answer:

- i. Dead node is a generated node that is not to be expended any further. In N-queen problem when any queen have the most possibility to be attacked by the positioning of others queens Dead node arise. By backtracking the problem can be solved.
- ii.



All possible arrangements for 4 non-attacking queens: 2,4,1,3.

5(c) P class: The P in the P class stands for Polynomial Time. It is the collection of decision problems (problems with a "yes" or "no" answer) that can be solved by a deterministic machine in polynomial time.

Features:

The solution to P problems is easy to find.

P is often a class of computational problems that are solvable and tractable.

NP class: The NP in NP class stands for Non-deterministic Polynomial Time. It is the collection of decision problems that can be solved by a non-deterministic machine in polynomial time.

Features:

The solutions of the NP class are hard to find since they are being solved by a non-deterministic machine but the solutions are easy to verify.

Problems of NP can be verified by a Turing machine in polynomial time.

NP-Complete class: A problem is NP-complete if it is both NP and NP-hard. NP-complete problems are the hard problems in NP.

Features:

NP-complete problems are special as any problem in NP class can be transformed or reduced into NP-complete problems in polynomial time.

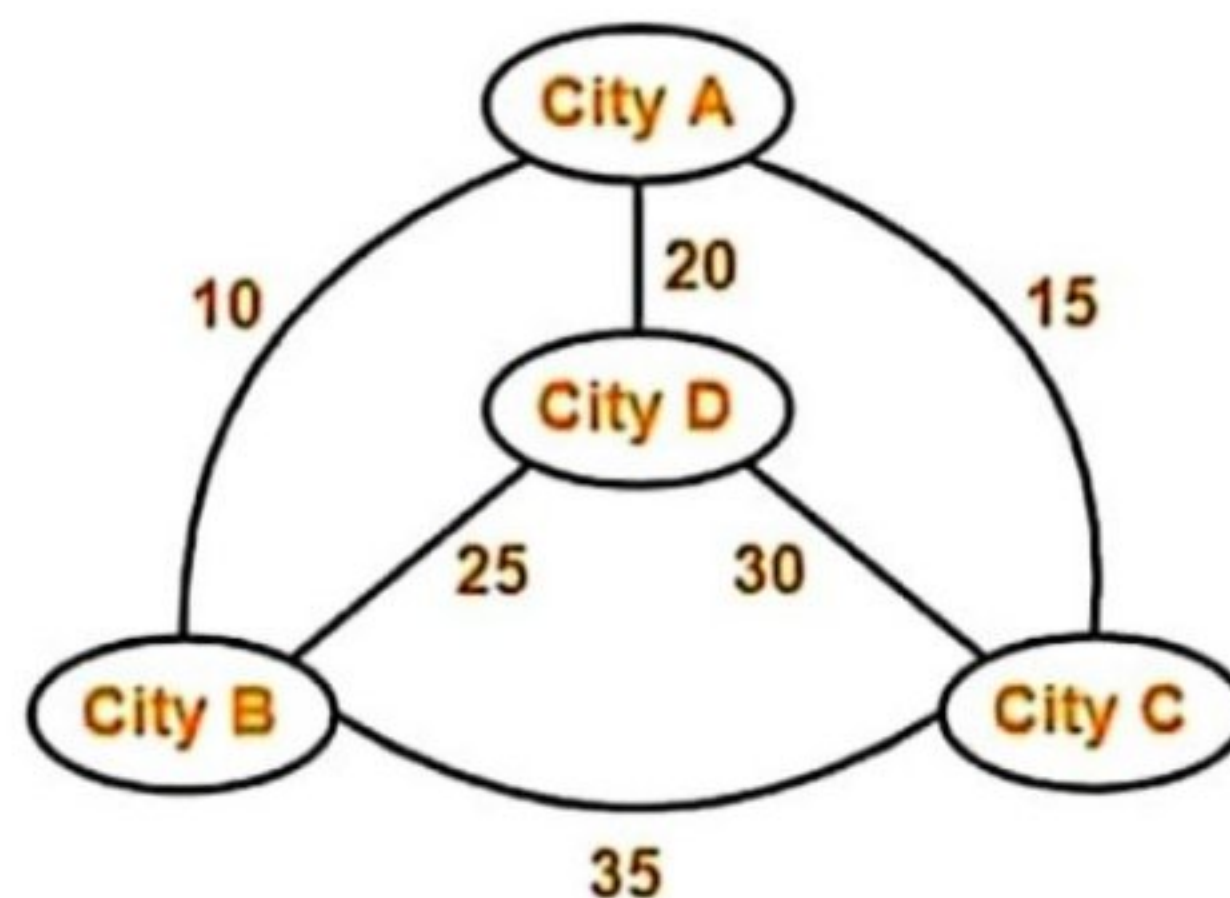
If one could solve an NP-complete problem in polynomial time, then one could also solve any NP problem in polynomial time.

5(c): What is the basic principle of a Branch-and-Bound algorithm? Explain how you will apply it in solving travelling salesman problem.

Answer:

Branch-and-Bound

- Effective for optimization problems
- Extended Backtracking Algorithm
- Instead of stopping once a single solution is found, continue searching until the best solution is found
- Has a scoring mechanism to choose most promising configuration in each iteration



Travelling Salesman Problem

If salesman starting city is A, then a TSP tour in the graph is-

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$

Cost of the tour

$= 10 + 25 + 30 + 15$

$= 80$ units

To reduce a matrix, perform the row reduction and column reduction of the matrix separately.

A row or a column is said to be reduced if it contains at least one entry '0' in it.